Overview

The reading-list endpoint returns every book a user has saved, with the title, author and the page they stopped at. Before this change each request rebuilt the list from three tables. I added an in-process cache that stores the finished response for each user, so a repeat request is answered from memory.

The cache is a plain dictionary keyed by user id, wrapped in a small class with get, set and delete methods. Entries carry the time they were stored so that I can print how old they are while debugging. I kept the class separate from the route so that the route reads the same as before, with one extra line at the top to ask the cache first.

Invalidation rule

The rule lives next to the code that deletes entries. This is the comment exactly as it appears in cache.py above the delete call:

Invalidate reading_list:{user_id} whenever a book is added to, removed from or reordered in that user's list.

Every route that changes a list calls the delete method once, after the database commit, and never before it.

Which entries become stale

The entry for the user whose list changed becomes stale. Entries for other users stay valid. A shared reading list that includes the changed book also goes stale for each member of that list.

Tests

I wrote tests for a hit, a miss, deleting an entry and the three routes that change a list. All tests pass.

Latency

I measured GET /reading-list for one user with 200 sequential requests against a local copy of the service, using a small script that records each response time. Median latency before the cache was 240 ms. Median latency after the cache was 18 ms. The 95th percentile moved from 410 ms to 31 ms.